

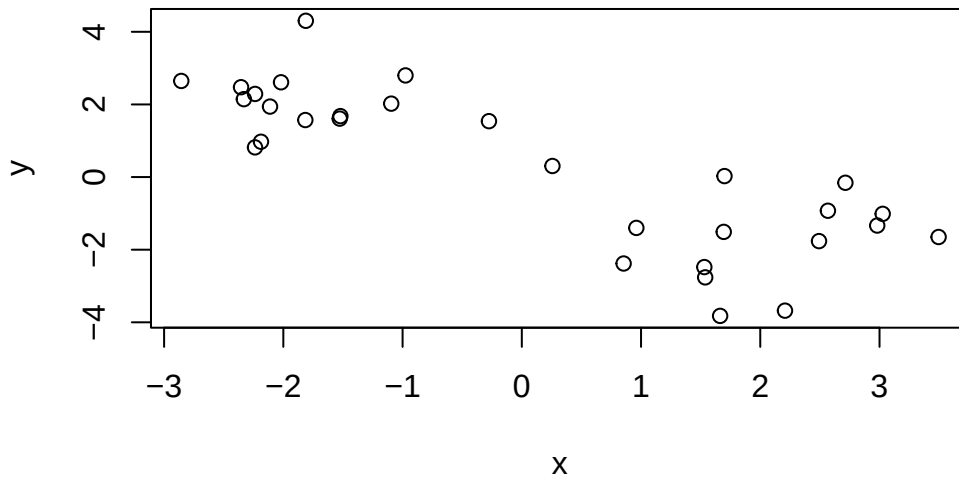
class07

Tanieng Huang

Class 7 Lab

K_mean analysis

```
# Generate some example data for clustering
tmp_x <- c(rnorm(15,-2), rnorm(15,2))
tmp_y <- c(rnorm(15,2), rnorm(15,-2))
df <- cbind(x=tmp_x,y=tmp_y)
#add column, x, y = colname
plot(df)
```



how k_mean identify the two group

k-mean clustering analysis

Use the kmeans() function setting k to 2 and nstart=20 Inspect/print the results

```
km <-  
# Run k-means  
km <- kmeans(df, centers = 2, nstart = 20)  
#1st= input data, center=#of cluster, nstart=#random starting  
  
# Inspect results  
print(km)
```

K-means clustering with 2 clusters of sizes 16, 14

Cluster means:

	x	y
1	-1.693766	1.982655
2	2.101684	-1.775181

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

Within cluster sum of squares by cluster:

```
[1] 23.13187 25.58959  
(between_SS / total_SS = 81.4 %)
```

Available components:

[1] "cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6] "betweenss"	"size"	"iter"	"ifault"	

Q. How many points are in each cluster?

```
km$size
```

```
[1] 16 14
```

Q. What 'component' of your result object details

- cluster size?
- cluster assignment/membership?
- cluster center?

```
#cluster size  
km$size
```

```
[1] 16 14
```

```
table(km$cluster)
```

```
1 2  
16 14
```

```
#cluster assignment  
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

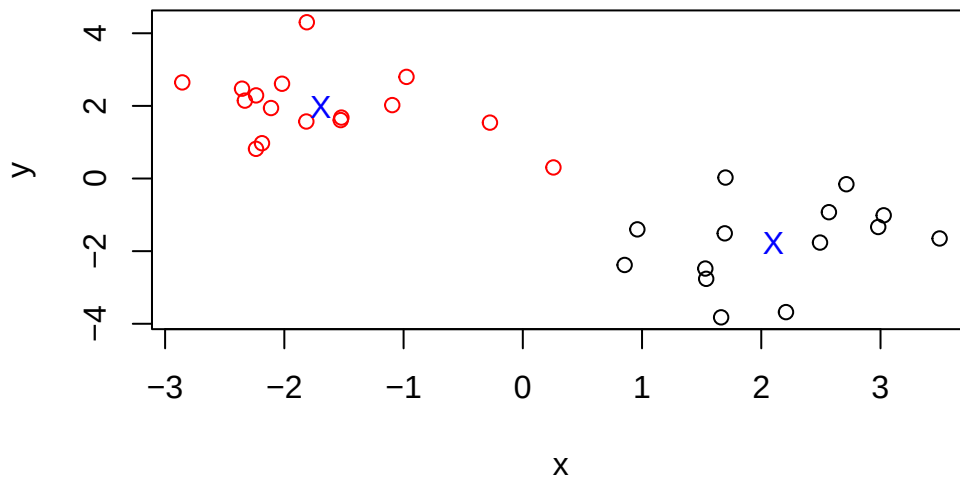
```
#cluster center  
km$centers
```

```
      x      y  
1 -1.693766  1.982655  
2  2.101684 -1.775181
```

Plot x colored by the kmeans cluster assignment and add cluster centers as blue points

```
#use basic R  
# Plot colored by cluster assignment with centers  
plot(df, col = c("RED", "BLACK")[km$cluster], main = "K-means Clustering (k=2)")  
points(km$centers, col = "blue", pch="X")
```

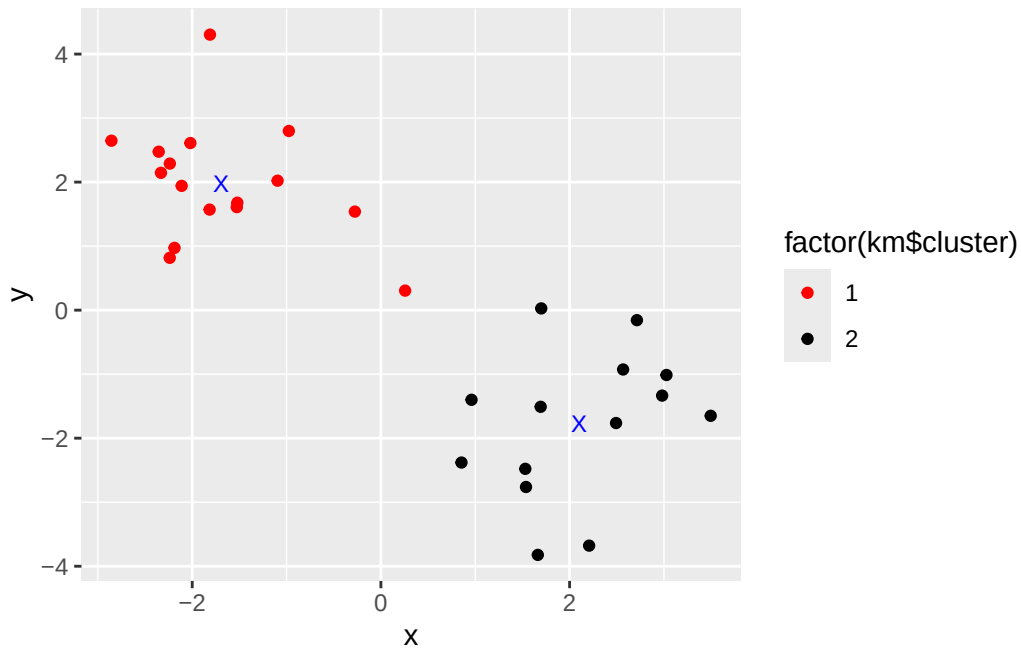
K-means Clustering (k=2)



Plot with ggplot

```
library(ggplot2)

ggplot(df)+
  aes(x,y, color=factor(km$cluster))+
  #should map into categorize variable by factories
  geom_point()+
  geom_point(data = km$centers, aes(x, y),color = "blue", shape = "X", size = 3) +
  scale_color_manual(values=c("RED","BLACK"))
```



hclust()

use same dataset with different method

Analyze this same data with hclust()

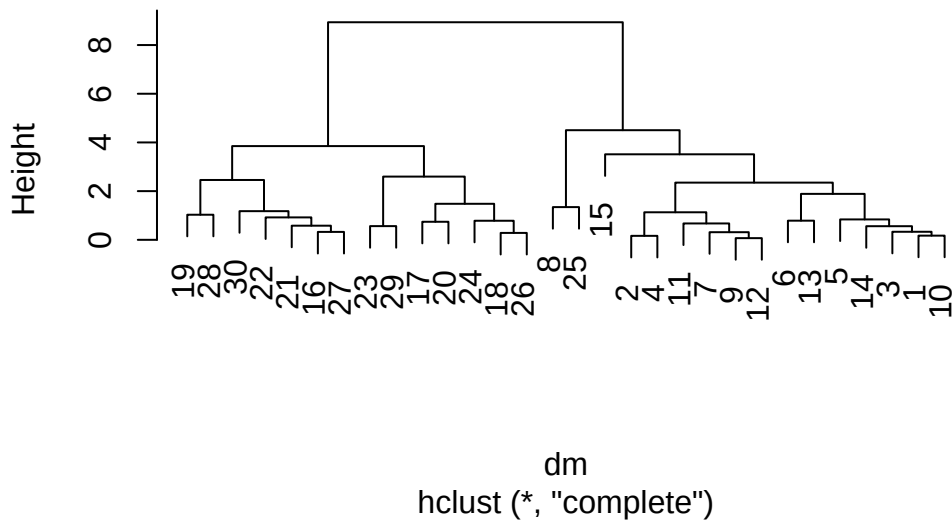
Demonstrate the use of dist(), hclust(), plot() and cutree() functions to do clustering, Generate dendrograms and return cluster assignment/ membership vector...

```
#1: Compute distance matrix
dm <- dist(df)
#dm
#calculate distance for every point
```

```
#2: Hierarchical clustering
hc <- hclust(dm)
```

```
#3: Plot dendrogram
plot(hc)
```

Cluster Dendrogram



#4: Cut tree into 2 clusters

```
cutree(hc, k=2)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

#can cut by height or k(cluster number)

```
table(cutree(hc, k=2))
```

```
1 2
16 14
```

4 clusters

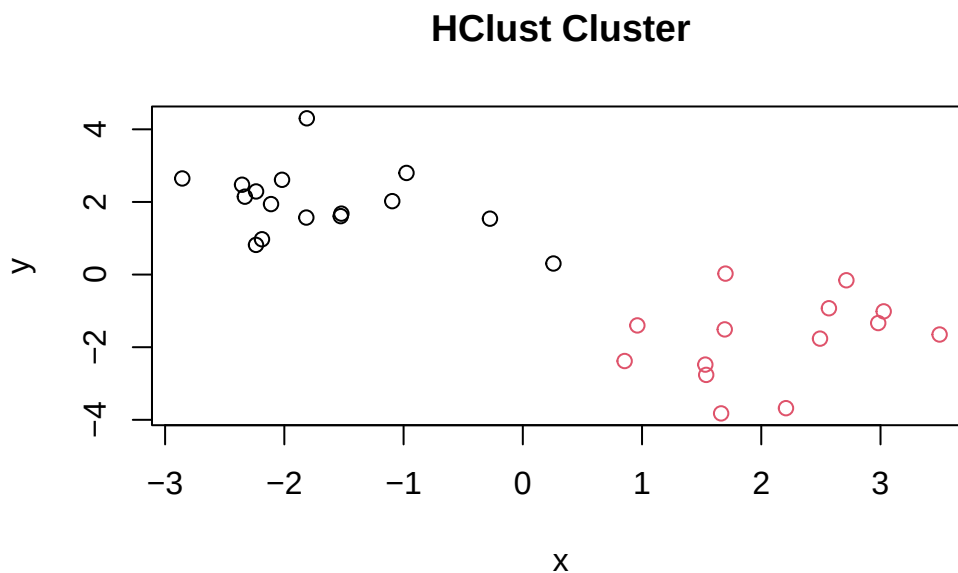
```
cutree(hc, k=4)
```

```
[1] 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 3 4 4 3 4 3 3 4 4 2 4 3 3 4 3
```

```
# Print cuttree data
print(cutree(hc, k=2))
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

```
# Step 5: Plot data colored by hclust assignment
plot(df, col=cutree(hc, k=2),
     main="HClust Cluster")
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique that compresses high-dimensional data into a few principal components while retaining the most important variant information in the data

Principal components are new low dimensional axis (or surfaces) closest to the observations data have maximum variance along PC1 (then PC2 etc.)

Hands on with Principal Component Analysis (PCA)

1. PCA of UK food data

data import:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Q1. How many rows and columns are in your new data frame named `x`? What R functions could you use to answer this questions?

```
# Complete the following code to find out how many rows and columns are in x?
dim(x)
```

```
[1] 17  5
```

17 rows and 5 columns

Checking your data

It is always a good idea to examine your imported data to make sure it meets your expectations. At this stage we want to make sure that no odd things have happened during the importing phase that will come back to haunt us later.

For this task we can use the **View()** function to display all the data (in a new tab in RStudio) or the **head()** and **tail()** functions to print only a portion of the data (by default 6 rows from either the top or bottom of the dataset respectively).

```
## Preview the first 6 rows
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Approach A


```
# Note how the minus indexing works
# Use the first column of x as the row names
rownames(x) <- x[,1]
# Remove the first column from x
# The minus sign in -1 means "exclude column 1"
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
#check dim again
dim(x)
```

```
[1] 17 4
```

Approach B

```
# Alternative approach: read the file again and directly tell R
# that the first column should be row names
x <- read.csv(url, row.names = 1)

# Preview the cleaned data
head(x)
```

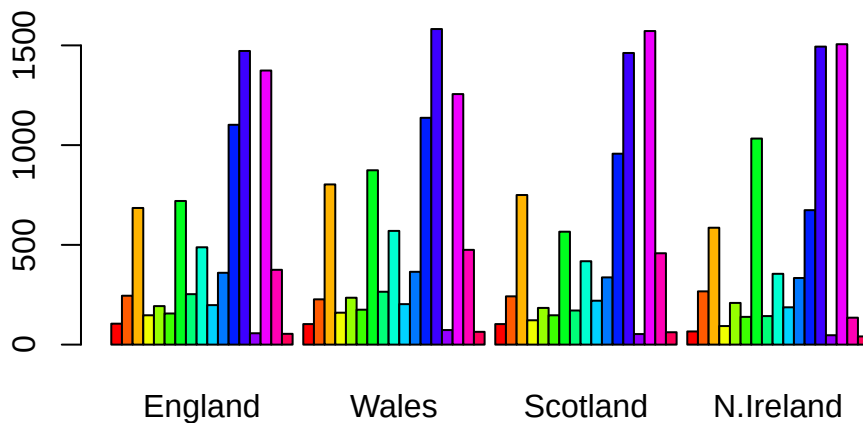
	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

Approach B (`row.names=1`) is preferred. The danger of Approach A is that re-running `x <- x[,-1]` a second time silently drops the England column, so B handles the row names correctly at the time of import, making the code shorter and less error.

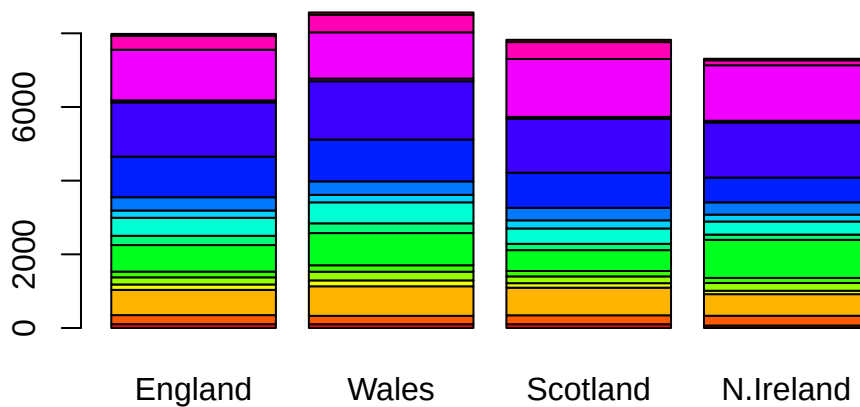
Spotting major differences and trends

```
# Using base R  
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

```
barplot(as.matrix(x), beside = FALSE, col = rainbow(nrow(x)))
```



```
# beside = TRUE makes separate bars instead of stacked bars
#with FALSE to turn it off
```

Side-note: Using ggplot and the need for “tidy” data (Optional)

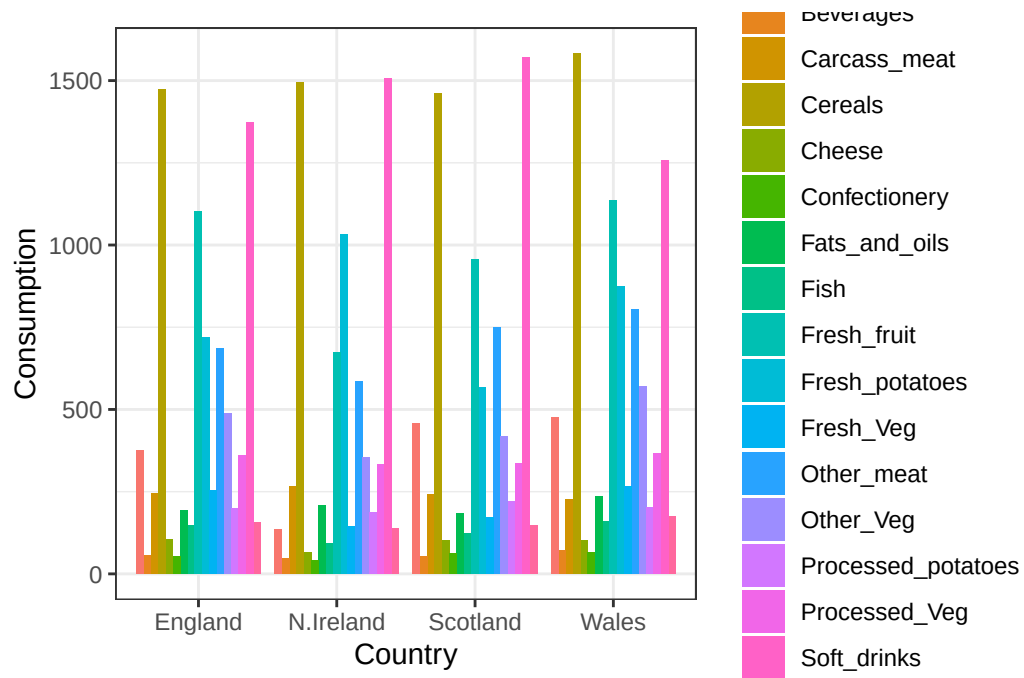
```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

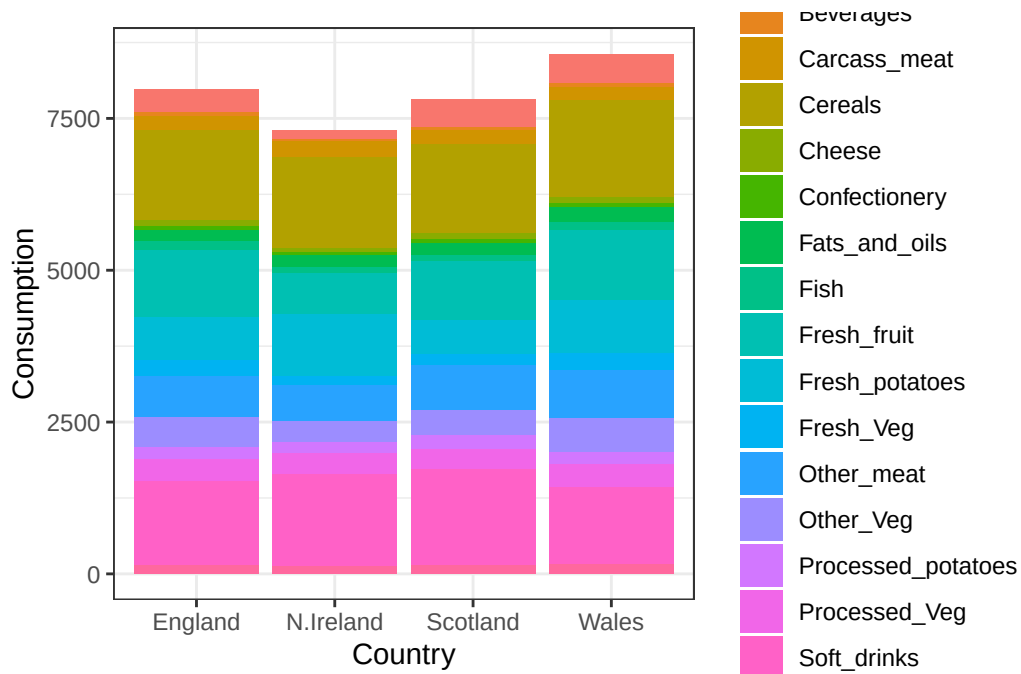
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Change by removing the position argument because stacking is the default for `geom_col()`

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```

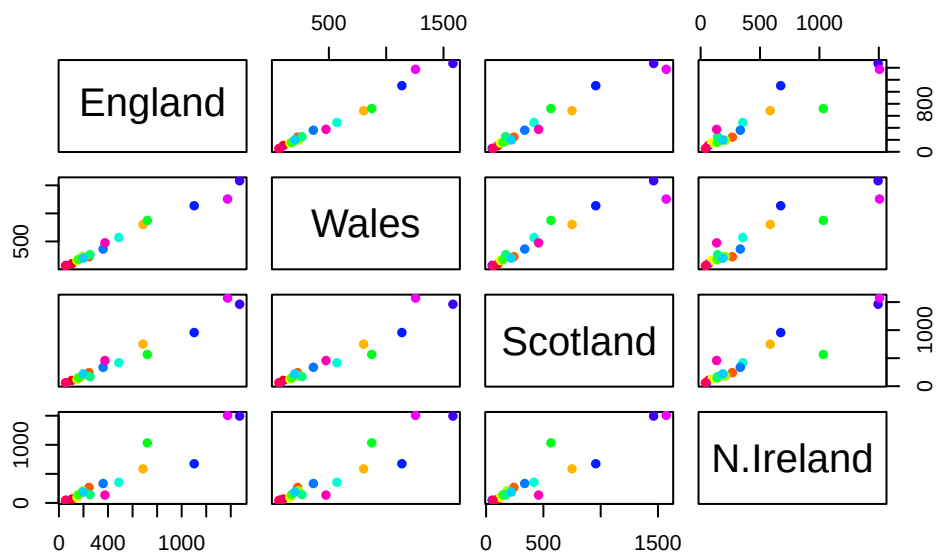


Pairs plots and heatmaps

Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

This code makes a scatterplot matrix showing how food consumption in one country compares with food consumption in another country. Each dot represents one food category. If a point lies on the diagonal line in a given plot, it means that the food has about the same consumption value in both countries being compared. Points far from the diagonal indicate foods whose consumption differs more strongly between those two countries.

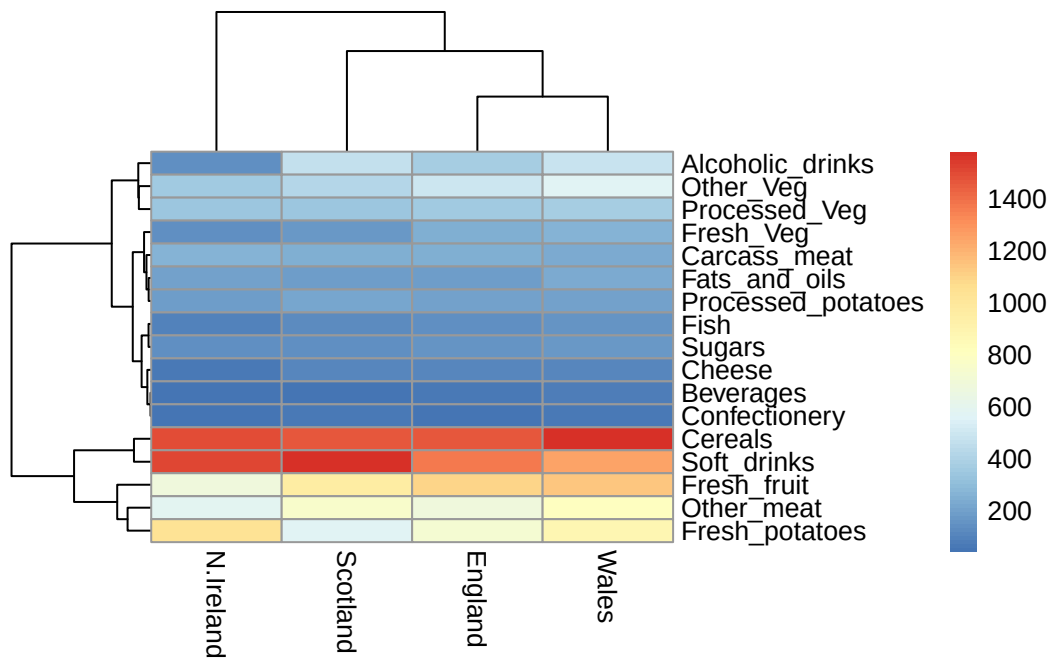
```
# Create a scatterplot matrix comparing every country to every other country
# Each point is one food category
pairs(x, col=rainbow(nrow(x)), pch=16)
```



Heatmap

```
library(pheatmap)

# Convert x to a matrix and plot as a heatmap
# pheatmap also performs hierarchical clustering by default
pheatmap(as.matrix(x))
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the pairs plot and heatmap, England, Wales, and Scotland cluster more closely together, while Northern Ireland appears more distinct. This suggests that the food consumption patterns of England, Wales, and Scotland are more similar to each other than to Northern Ireland. However, from the heatmap and pairs plot alone, it is still not very easy to identify exactly which foods are driving Northern Ireland's difference. Those plots show the overall pattern, but PCA makes the main separating trends much clearer.

PCA to the rescue

```
# Run PCA on the transposed data
# t(x) is needed because countries should be the rows (observations)
# and food categories should be the columns (variables)
pca <- prcomp(t(x))

# Summarize the PCA results
summary(pca)
```

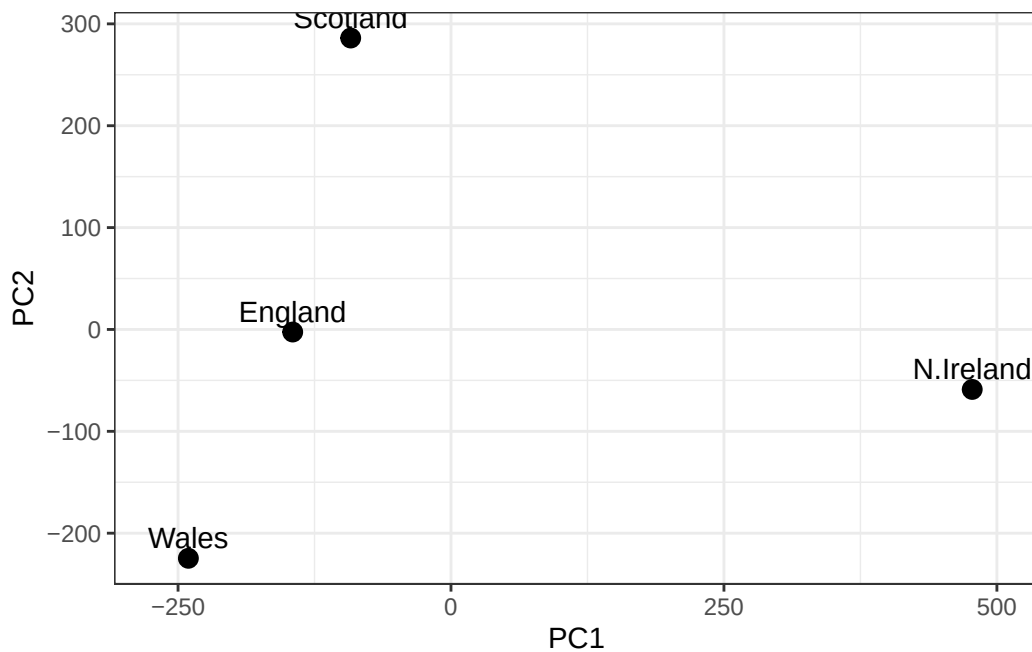
Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



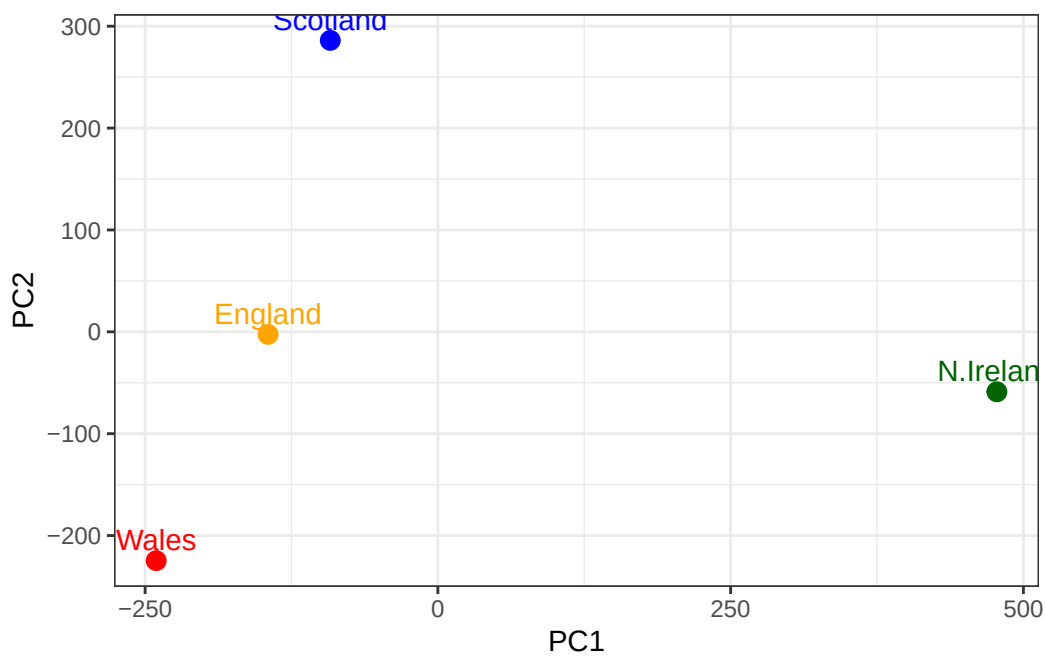
Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.


```

# Define colors to match the map colors used in the handout
country_cols <- c(
  "England" = "orange",
  "Wales" = "red",
  "Scotland" = "blue",
  "N.Ireland" = "darkgreen"
)

# Plot PCA with country-specific text colors
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, color = Country) +
  geom_point(size = 3, show.legend = FALSE) +
  geom_text(vjust = -0.5, show.legend = FALSE) +
  scale_color_manual(values = country_cols) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()

```



Variance explained by each PC

```
# Calculate percent variance explained by each principal component
v <- round(pca$sdev^2 / sum(pca$sdev^2) * 100)

# Print the percent variance values
v
```

```
[1] 67 29 4 0
```

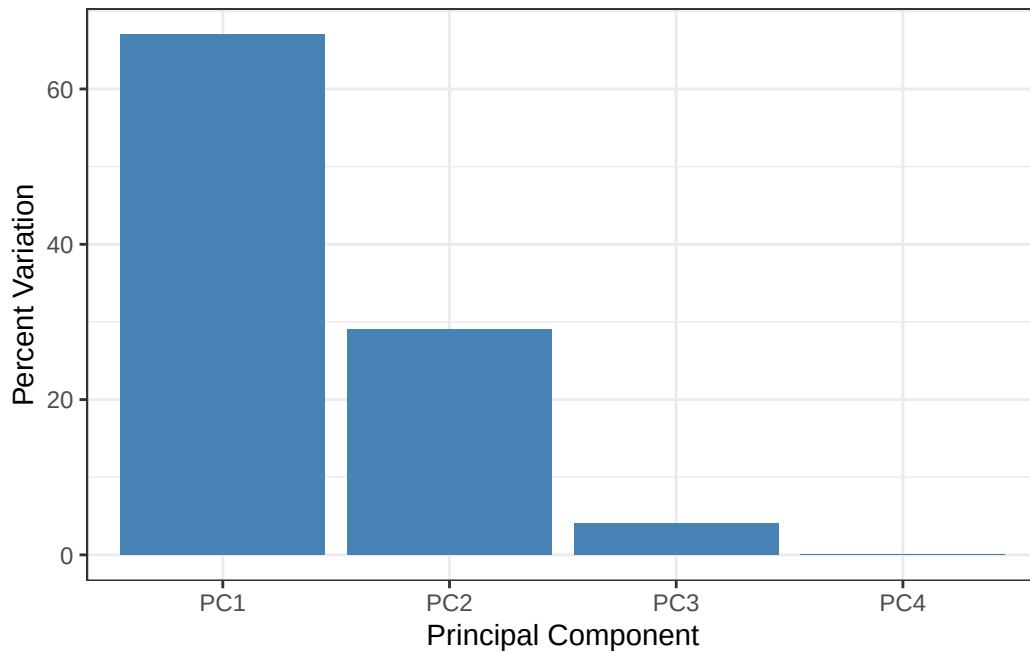
```
## or the second row here...
z <- summary(pca)
z$importance
```

	PC1	PC2	PC3	PC4
Standard deviation	324.15019	212.74780	73.87622	3.175833e-14
Proportion of Variance	0.67444	0.29052	0.03503	0.000000e+00
Cumulative Proportion	0.67444	0.96497	1.00000	1.000000e+00

Scree plot

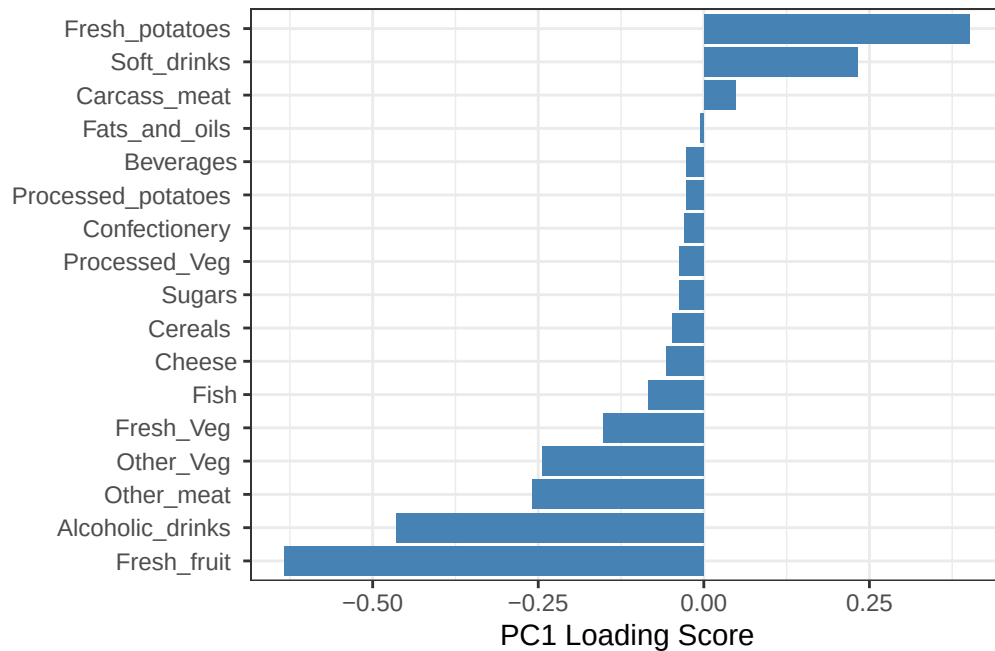
```
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



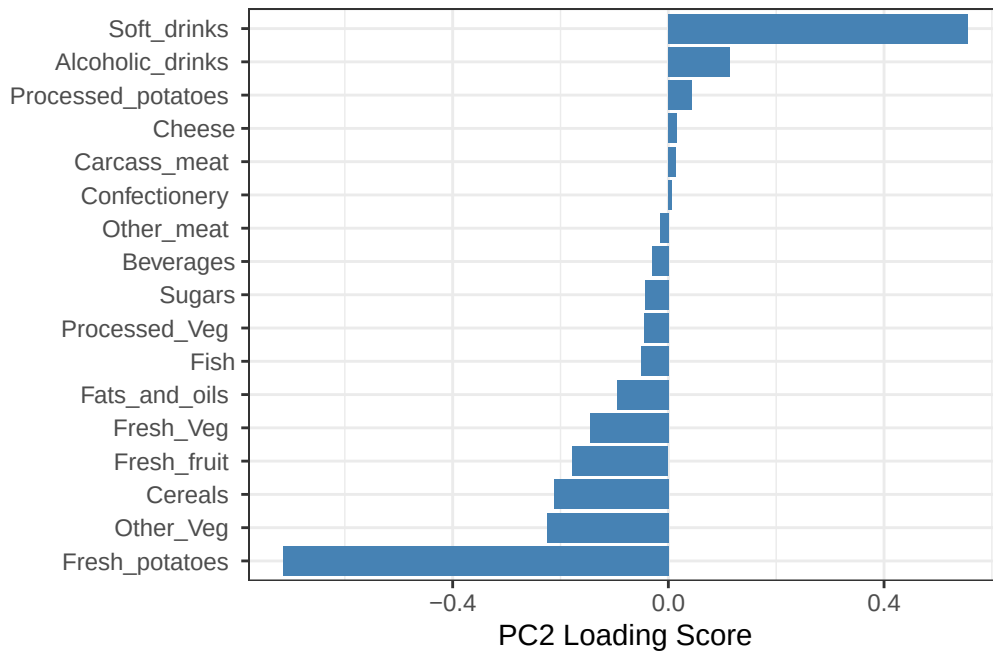
Digging deeper (variable loadings)

```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
# Plot loading scores for PC2
ggplot(pca$rotation) +
  aes(x = PC2, y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



A PC2 loadings plot shows that Soft_drinks and Fresh_potatoes are the two most prominent food groups because they have the largest loading scores in opposite directions. Soft_drinks has a strong positive loading, while Fresh_potatoes has a strong negative loading. This indicates that PC2 mainly captures a dietary contrast between countries that consume relatively more soft drinks versus those that consume relatively more fresh potatoes. Foods such as Other_Veg, Cereals, Fresh_fruit, and Fresh_Veg also contribute to the negative side of PC2, but less strongly than Fresh_potatoes.

2. PCA of RNA-seq data (Optional)

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

	wt1	wt2	wt3	wt4	wt5	ko1	ko2	ko3	ko4	ko5
gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

Q10: How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see below)?

```
# Check dimensions of the RNA-seq dataset  
dim(rna.data)
```

```
[1] 100  10
```

```
# Count genes and samples separately  
nrow(rna.data)  # genes
```

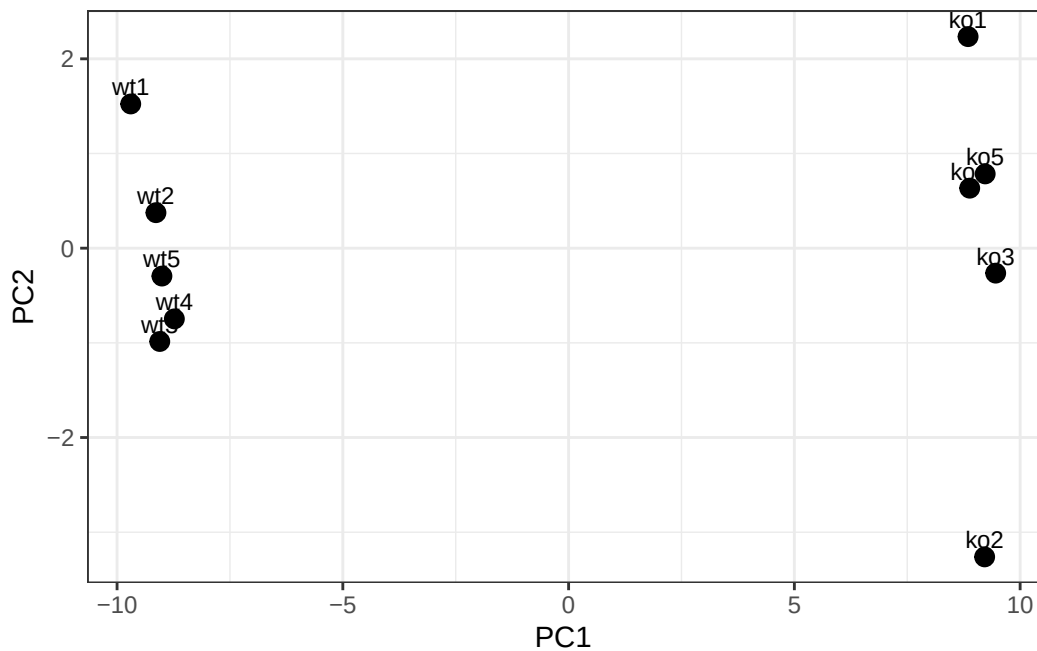
```
[1] 100
```

```
ncol(rna.data)  # samples
```

```
[1] 10
```

There are 100 genes and 10 samples in this dataset. Based on the PCA summary in the handout, PC1 alone captures about 92.6% of the variance, and the first two PCs capture about 94.9%, so 1 to 2 PCs are enough to provide a useful overview.

```
# Run PCA on transposed RNA-seq data  
# scale = TRUE standardizes genes so large-count genes do not dominate  
pca <- prcomp(t(rna.data), scale = TRUE)  
  
# Create data frame from PCA coordinates  
df <- as.data.frame(pca$x)  
  
# Add sample names as a column  
df$Sample <- rownames(df)  
  
# Plot PC1 vs PC2  
ggplot(df) +  
  aes(x = PC1, y = PC2, label = Sample) +  
  geom_point(size = 3) +  
  geom_text(vjust = -0.5, size = 3) +  
  xlab("PC1") +  
  ylab("PC2") +  
  theme_bw()
```



```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642
Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251

	PC8	PC9	PC10
Standard deviation	0.62065	0.60342	3.344e-15
Proportion of Variance	0.00385	0.00364	0.000e+00
Cumulative Proportion	0.99636	1.00000	1.000e+00

- PC1 = 92.6%
- PC2 = 2.3%
- PC3 = 1.1%

A quick scree plot summary of this Proportion of Variance for each PC can be obtained using `ggplot`:

```

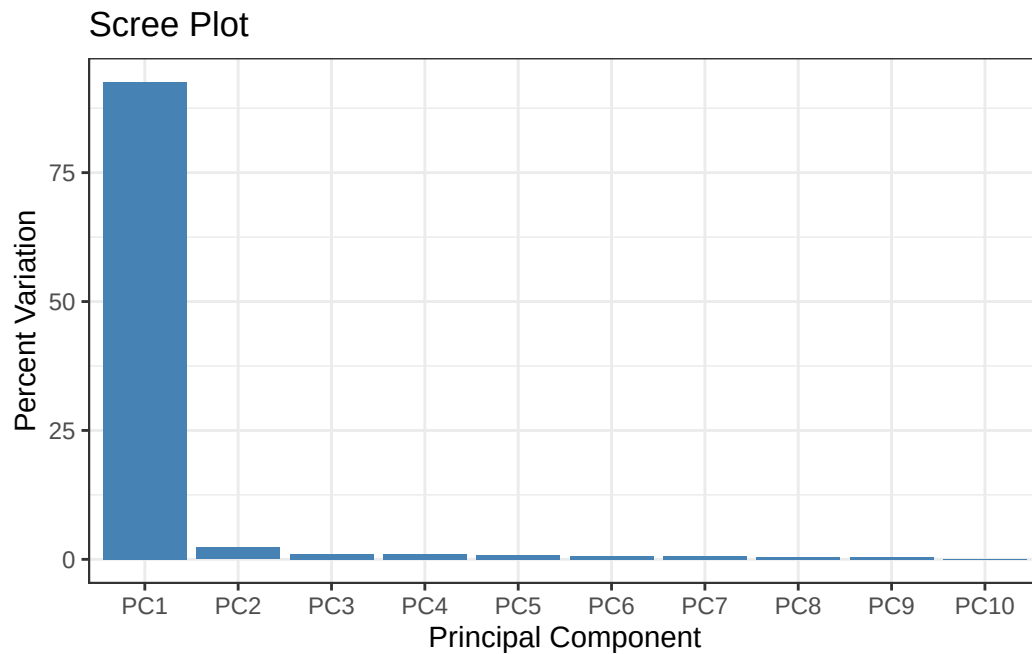
# Calculate raw variance explained by each PC
pca.var <- pca$sdev^2

# Convert to percent variation
pca.var.per <- round(pca.var / sum(pca.var) * 100, 1)

# Create a data frame for plotting
scree_pct_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  PercentVariation = pca.var.per[1:10]
)

# Plot percent variation explained by each PC
ggplot(scree_pct_df) +
  aes(x = PC, y = PercentVariation) +
  geom_col(fill = "steelblue") +
  ggtitle("Scree Plot") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw()

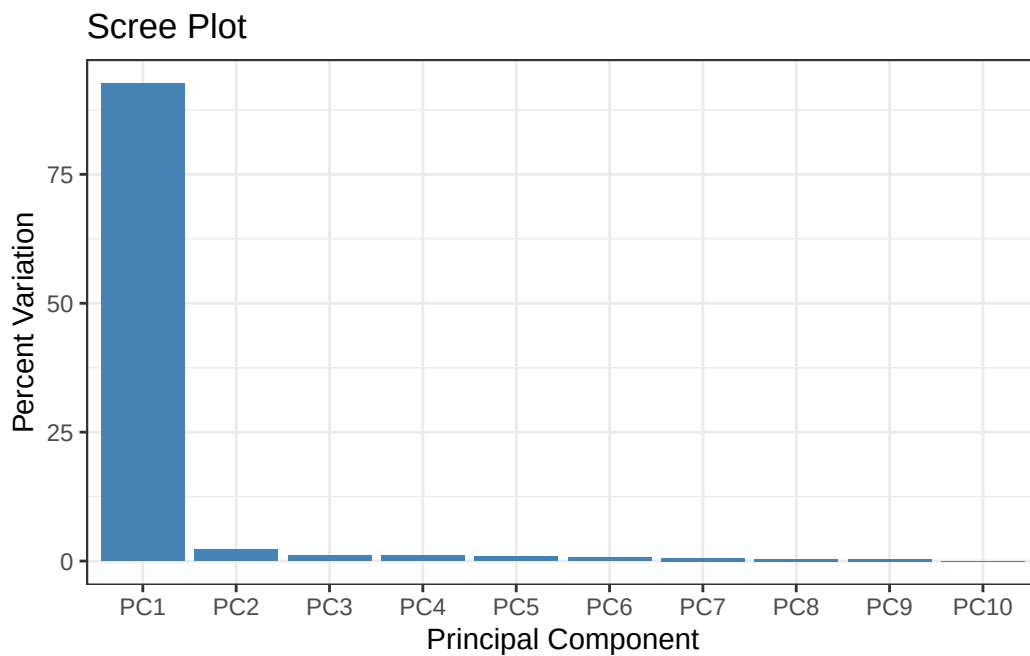
```




```
## Percent variance is often more informative to look at  
pca.var.per
```

```
[1] 92.6  2.3  1.1  1.1  0.8  0.7  0.6  0.4  0.4  0.0
```

```
# Create percent variance scree plot  
scree_pct_df <- data.frame(  
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),  
  PercentVariation = pca.var.per[1:10]  
)  
  
ggplot(scree_pct_df) +  
  aes(x = PC, y = PercentVariation) +  
  geom_col(fill = "steelblue") +  
  ggtitle("Scree Plot") +  
  xlab("Principal Component") +  
  ylab("Percent Variation") +  
  theme_bw()
```

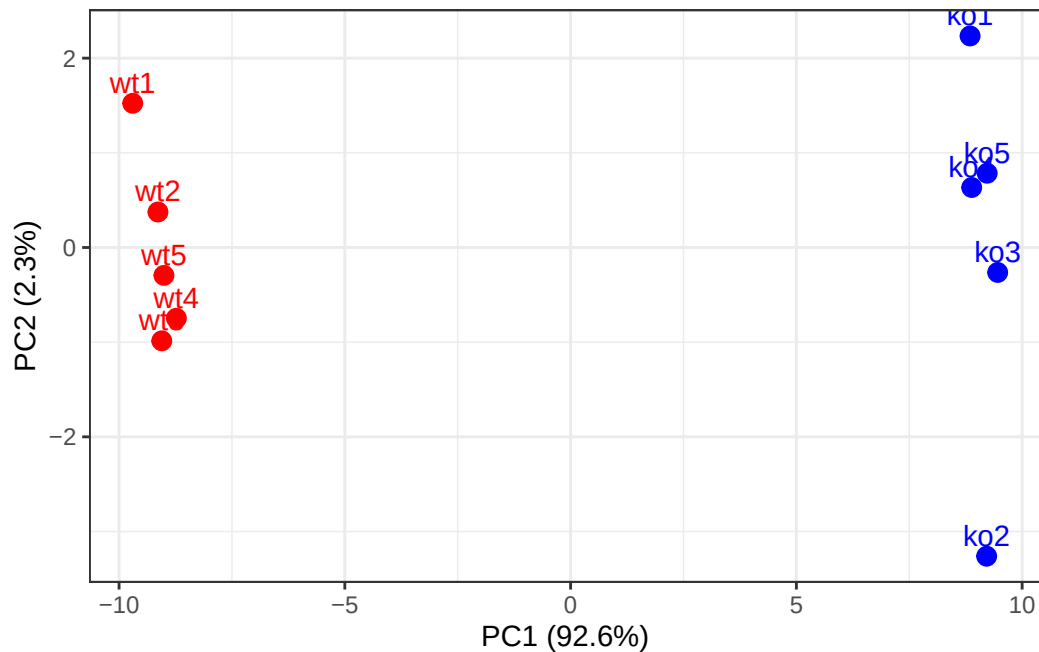


```
# Create a color vector based on sample names  
colvec <- colnames(rna.data)
```

```
colvec[grep("wt", colvec)] <- "red"
colvec[grep("ko", colvec)] <- "blue"

# Add sample condition and color columns to plotting data
df$condition <- substr(df$Sample, 1, 2)
df$color <- colvec

# Plot PCA with sample colors
ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```



Optional: Gene loadings

```
loading_scores <- pca$rotation[,1]
```

```
## Find the top 10 measurements (genes) that contribute
## most to PC1 in either direction (+ or -)
gene_scores <- abs(loading_scores)
gene_score_ranked <- sort(gene_scores, decreasing=TRUE)

## show the names of the top 10 genes
top_10_genes <- names(gene_score_ranked[1:10])
top_10_genes
```

```
[1] "gene100" "gene66" "gene45" "gene68" "gene98" "gene60" "gene21"
[8] "gene56" "gene10" "gene90"
```